Universidad Autónoma Metropolitana
Unidad Cuajimalpa

Integración de Sistemas

Microservicios para E-commerce

Proyecto Final

Integrantes:
Aaron Rodrigo Ramos Reyes
Eduardo Hernández Soto
Diego Sebastian Reyes García

Cuajimalpa, Ciudad de México
30 de marzo de 2026

Índice

1. Introducción **2**

2. Descripción del Proyecto **2**

3. Servicios Implementados **2**
3.1. 1. User Service (Puerto 8001) 2
3.2. 2. Product Service (Puerto 8002) 2
3.3. 3. Cart Service (Puerto 8003) 2
3.4. 4. Order Service (Puerto 8004) 2
3.5. 5. Shipment Service (Puerto 8005) 2

4. Exposición de Servicios **3**

5. Consumo de Servicios entre Sistemas **3**

6. Funcionamiento Correcto de la Integración **3**

7. Funcionamiento del Sistema y Frontend **4**
7.1. Interfaz del Frontend 4
7.2. Flujo Completo de Compra 5
7.3. Evidencia de Integración 6

8. Diagrama de Clases **9**

9. Diagrama de Flujo **10**

10. Arquitectura del Sistema **10**

11. Aprendizajes **11**

12. Conclusiones **11**

1. Introducción

El presente proyecto consiste en el desarrollo de una aplicación de e-commerce utilizando una arquitectura de microservicios. El sistema está compuesto por cinco servicios independientes que se comunican entre sí mediante APIs REST y comparten información a través de una red Docker.

El objetivo principal es demostrar la integración de múltiples servicios backend con un frontend simple, implementando un flujo completo de compra: registro de usuarios, catálogo de productos, carrito de compras, generación de órdenes y gestión de envíos.

2. Descripción del Proyecto

El sistema "EcoShop" es una plataforma de comercio electrónico minimalista que permite a los usuarios navegar por un catálogo de productos, agregar artículos al carrito, generar órdenes de compra y rastrear sus envíos.

La arquitectura está basada en microservicios, lo que permite escalabilidad, mantenimiento independiente y mayor resiliencia del sistema.

3. Servicios Implementados

3.1. 1. User Service (Puerto 8001)

Responsable de la gestión de usuarios, registro, inicio de sesión y autenticación mediante JWT. Es el servicio base que proporciona el token necesario para acceder a los demás servicios.

3.2. 2. Product Service (Puerto 8002)

Servicio de solo lectura que maneja el catálogo de productos. Incluye productos de ejemplo (laptops, teléfonos, accesorios) y proporciona información como nombre, descripción, precio y stock.

3.3. 3. Cart Service (Puerto 8003)

Gestiona el carrito de compras por usuario. Permite agregar productos, actualizar cantidades, eliminar items y vaciar el carrito completo. Almacena el precio del producto en el momento de agregar al carrito.

3.4. 4. Order Service (Puerto 8004)

Orquestador principal del flujo de compra. Se encarga de:

- Obtener el carrito del usuario
- Crear la orden de compra
- Guardar los items de la orden
- Vaciar el carrito
- Llamar al servicio de envíos

3.5. 5. Shipment Service (Puerto 8005)

Gestiona las órdenes de envío. Crea automáticamente un envío cuando se genera una orden, asignando un número de tracking y manteniendo el estado del envío (pendiente, en camino, entregado).

4. Exposición de Servicios

Cada microservicio expone una serie de endpoints (recursos y operaciones) a través de una API RESTful, los cuales pueden ser consumidos tanto por el frontend como por otros microservicios.

- **User Service (8001)**: Expone operaciones de registro, login y validación de usuarios mediante JWT.
- **Product Service (8002)**: Expone endpoints de consulta de catálogo de productos (solo lectura).
- **Cart Service (8003)**: Expone operaciones completas de gestión del carrito (agregar, actualizar, eliminar y vaciar).
- **Order Service (8004)**: Expone la creación de órdenes de compra y consulta de historial.
- **Shipment Service (8005)**: Expone la creación y consulta de órdenes de envío.

Todos los servicios utilizan el estándar OpenAPI (Swagger) para documentar sus endpoints, accesibles en la ruta `/docs` de cada servicio.

5. Consumo de Servicios entre Sistemas

Uno de los requisitos fundamentales del proyecto es la **comunicación bidireccional** entre microservicios. El sistema no funciona de forma aislada, sino que los servicios se consumen mutuamente.

- El **Cart Service** consume el **Product Service** para obtener el precio real del producto al momento de agregarlo al carrito.
- El **Order Service** consume el **Cart Service** para:
  - Obtener el contenido actual del carrito del usuario.
  - Vaciar el carrito una vez generada la orden.
- El **Order Service** consume el **Shipment Service** para crear automáticamente una orden de envío al generar una orden de compra.
- El **Frontend** consume todos los servicios mediante peticiones HTTP autenticadas con JWT.

Esta integración asegura que los servicios no sean independientes, sino que formen un ecosistema interconectado y funcional.

6. Funcionamiento Correcto de la Integración

La integración entre los microservicios se ejecuta correctamente, demostrando una comunicación efectiva entre sistemas.

El flujo completo de compra funciona de la siguiente manera:

1. El usuario se autentica en el **User Service** y obtiene un token JWT.
2. Consulta el catálogo a través del **Product Service**.

3. Agrega productos al carrito mediante el **Cart Service**.
4. Al generar la orden, el **Order Service**:
   - Consulta el carrito actual.
   - Crea la orden y sus ítems correspondientes.
   - Vacía el carrito.
   - Solicita la creación automática de un envío al **Shipment Service**.
5. El usuario puede consultar sus órdenes y envíos en tiempo real.

Se ha verificado que:

- La comunicación entre servicios se realiza correctamente mediante HTTP.
- El manejo de autenticación (JWT) funciona entre servicios.
- Las transacciones entre servicios (creación de orden + envío) se ejecutan de forma coordinada.
- El frontend puede consumir todos los endpoints de manera exitosa.

La integración se considera funcional en su totalidad, cumpliendo con los objetivos de exposición y consumo de servicios entre sistemas.

7. Funcionamiento del Sistema y Frontend

Una vez implementada la integración entre los microservicios y el frontend, se procedió a verificar el correcto funcionamiento del sistema completo. A continuación se presenta evidencia visual y descriptiva del comportamiento observado.

7.1. Interfaz del Frontend

El frontend desarrollado con HTML, JavaScript y Tailwind CSS permite una interacción fluida con todos los servicios backend. El usuario puede navegar por el catálogo, gestionar su carrito y visualizar sus órdenes y envíos de manera intuitiva.

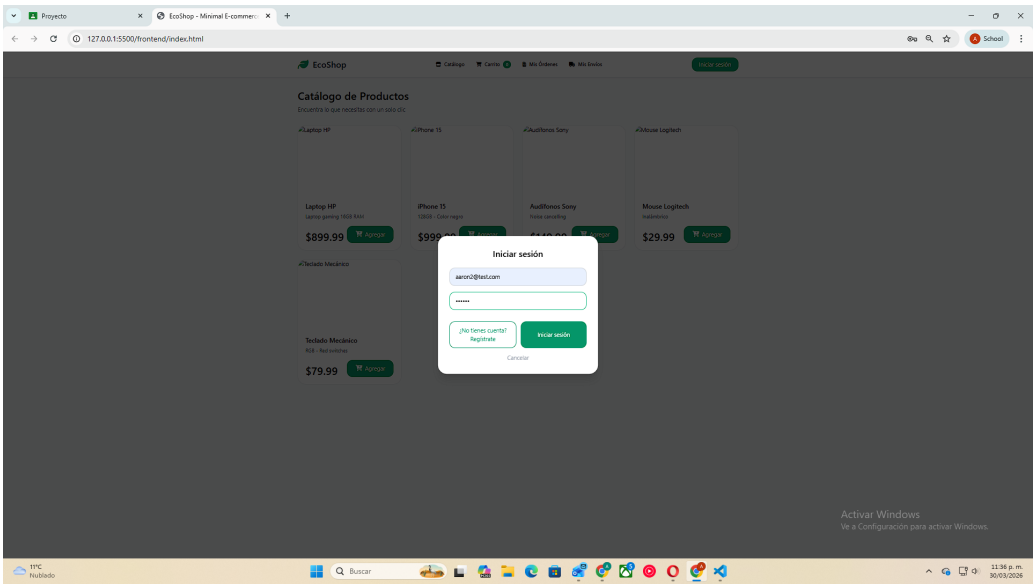

Figura 1: Pantalla de inicio de sesión y registro

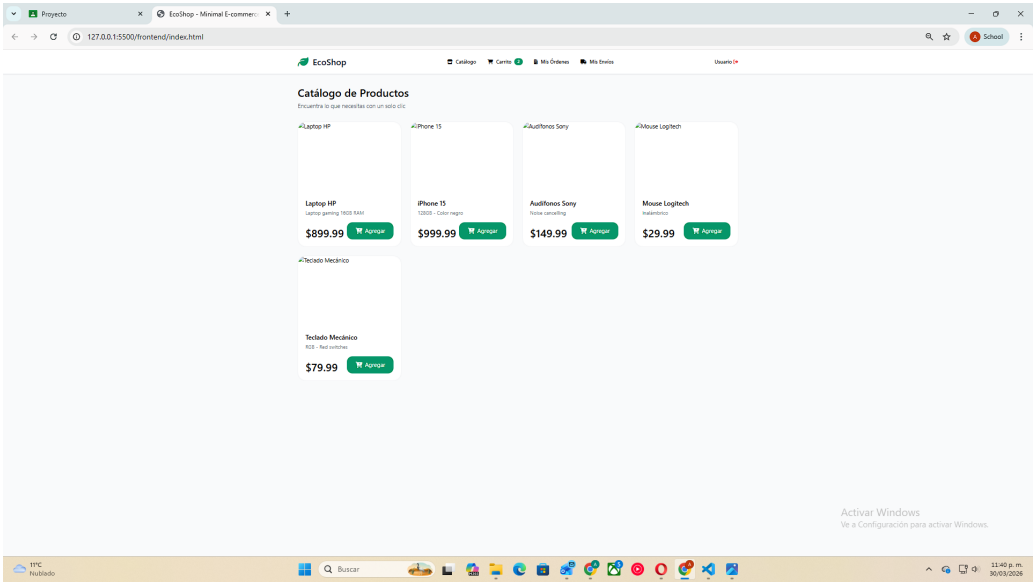

Figura 2: Catálogo de productos cargado correctamente desde el Product Service

7.2. Flujo Completo de Compra

Se validó exitosamente el flujo completo de una compra:

- Inicio de sesión y obtención de token JWT.
- Navegación por el catálogo y adición de productos al carrito.
- Visualización y modificación del carrito de compras.
- Generación exitosa de una orden de compra.
- Creación automática del envío asociado.

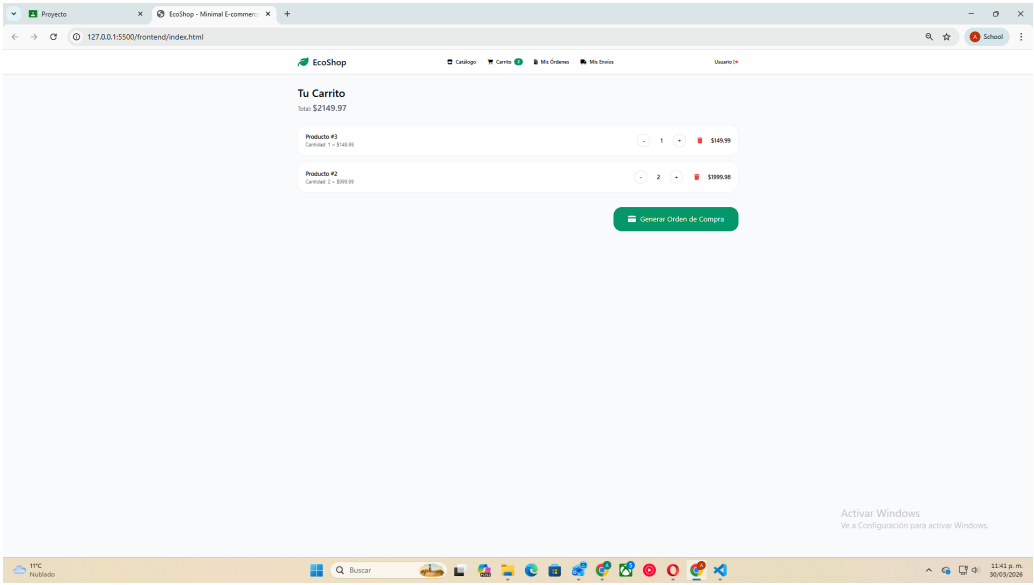

Figura 3: Carrito de compras con productos agregados

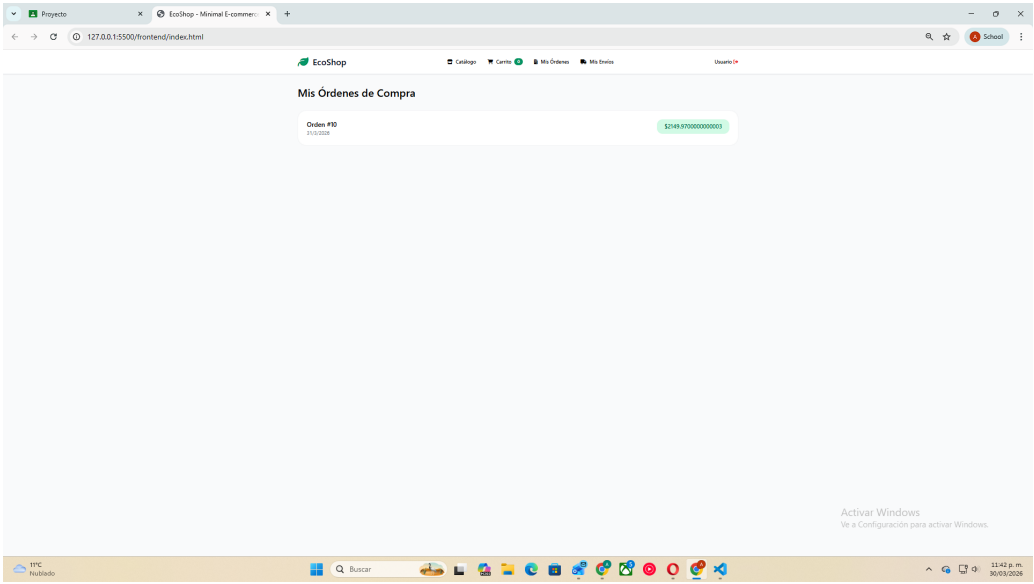

Figura 4: Confirmación de orden generada correctamente

7.3. Evidencia de Integración

Las siguientes capturas muestran el correcto funcionamiento de las secciones "Mis Órdenesz "Mis Envíos", demostrando que la comunicación entre el Order Service y el Shipment Service se ejecuta de forma adecuada.

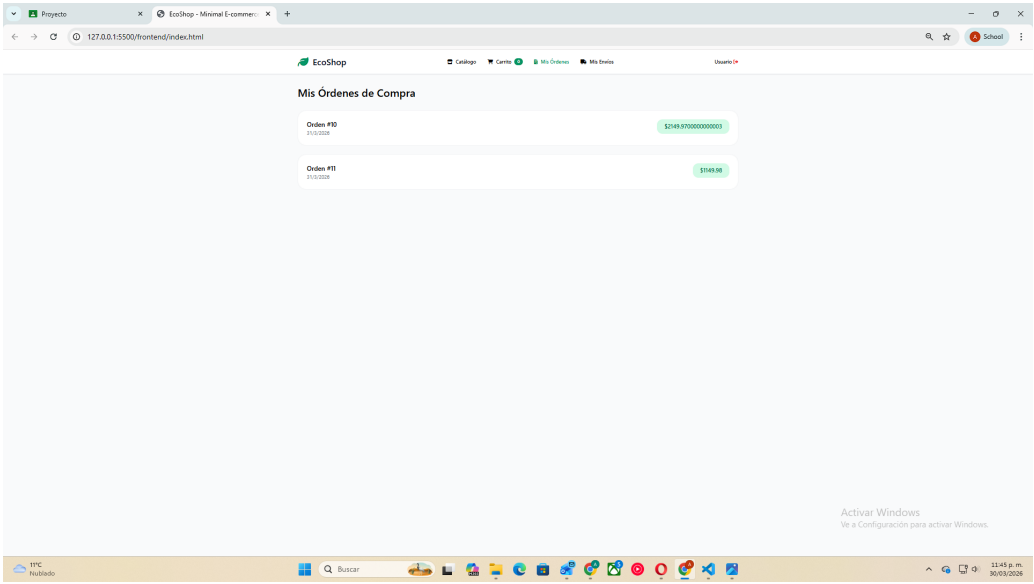

Figura 5: Pantalla de Mis Órdenes mostrando las órdenes generadas

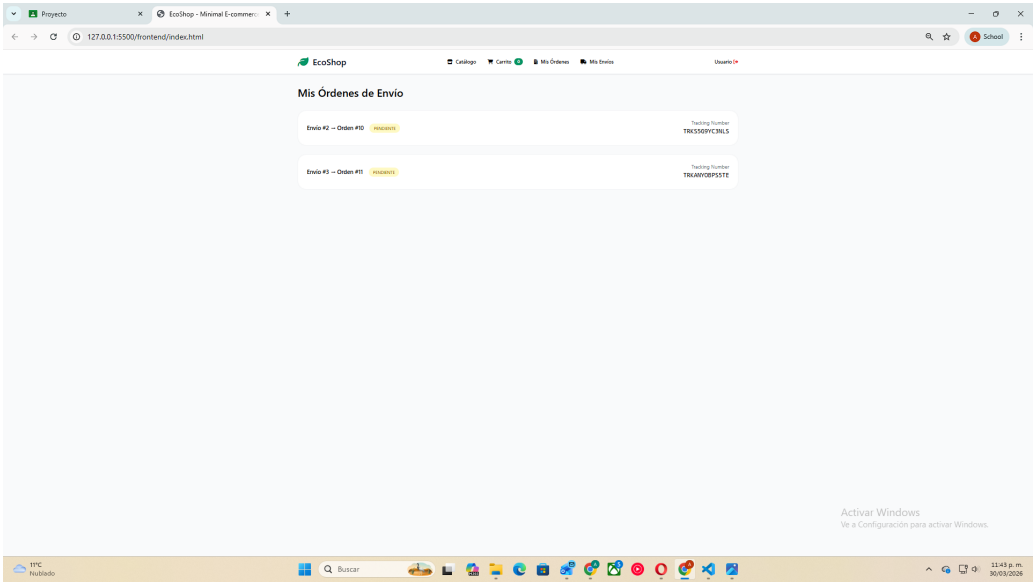

Figura 6: Pantalla de Mis Envíos con los envíos creados automáticamente

El sistema demuestra una integración estable y funcional entre el frontend y los cinco microservicios, cumpliendo con los requisitos de exposición, consumo y comunicación bidireccional entre servicios.

8. Diagrama de Clases

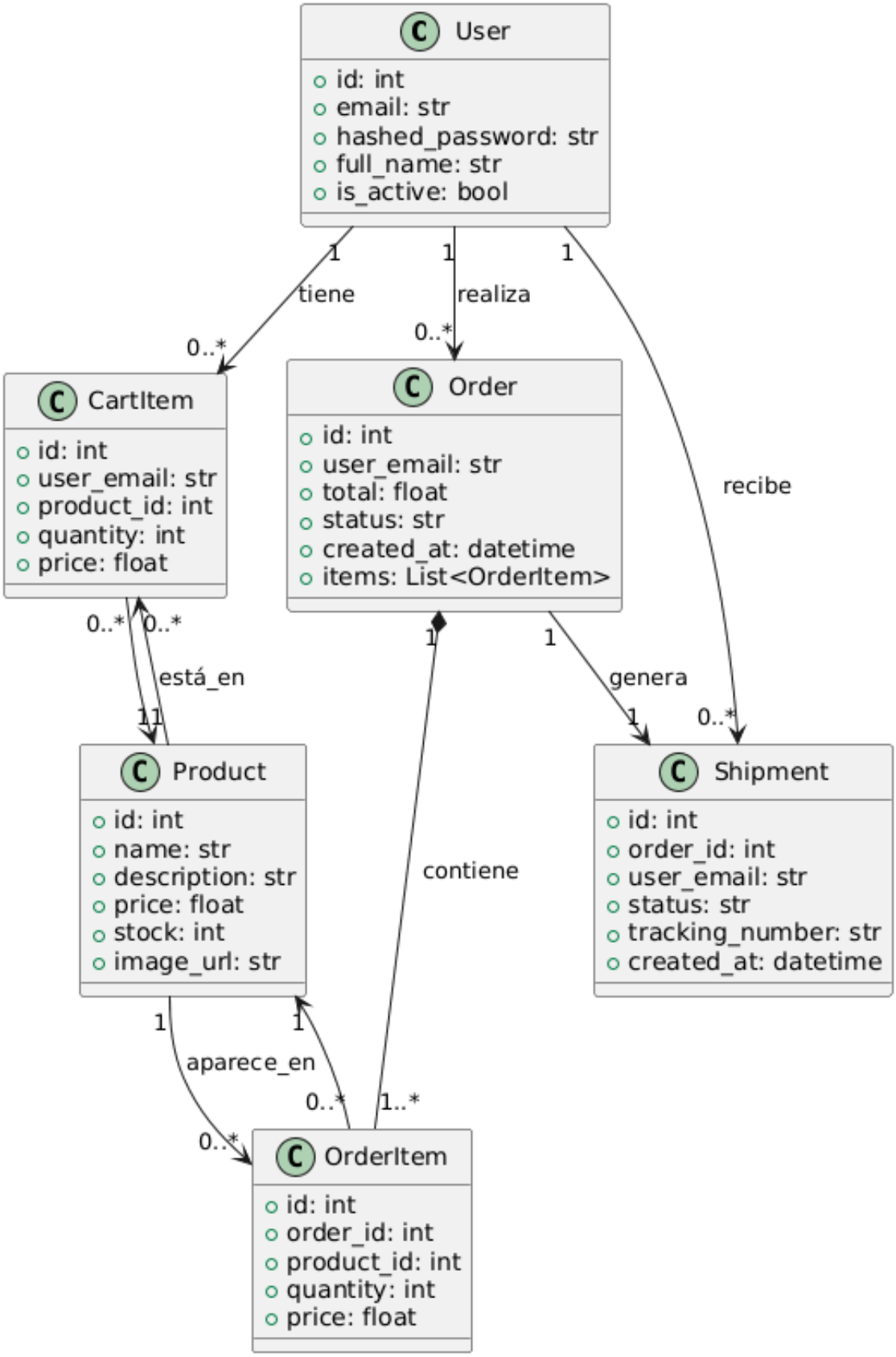

Figura 7: Diagrama de Clases del sistema de microservicios

9. Diagrama de Flujo

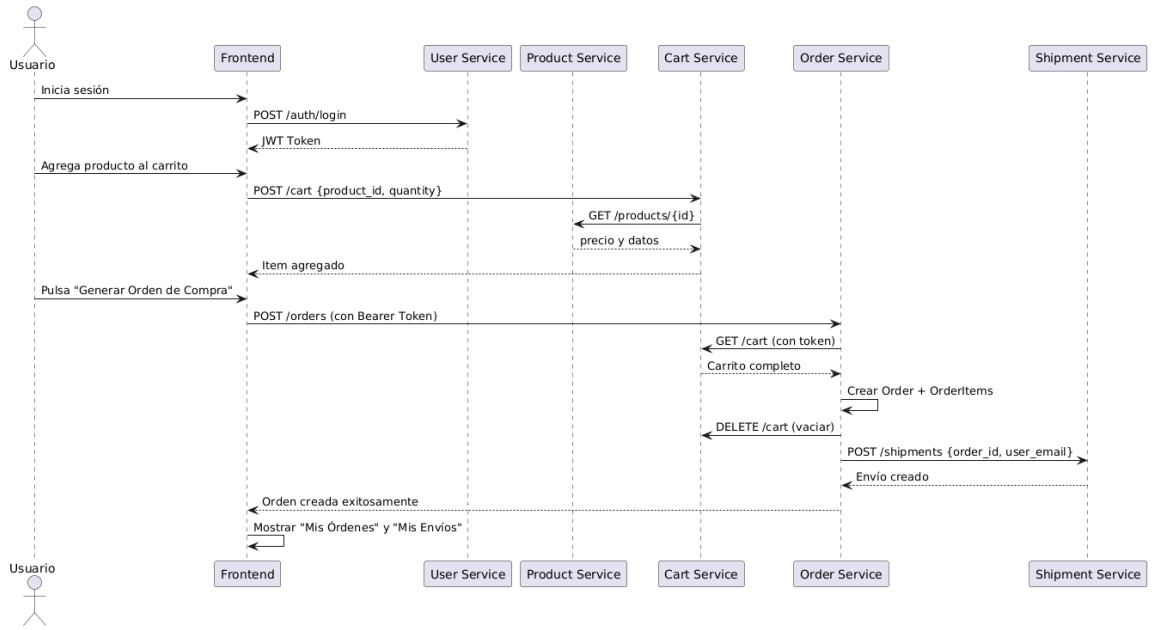

Figura 8: Diagrama de Secuencia - Flujo de Compra

El flujo principal comienza con la autenticación del usuario, continúa con la selección de productos, agregarlos al carrito, y culmina con la creación de la orden y el envío automático.

10. Arquitectura del Sistema

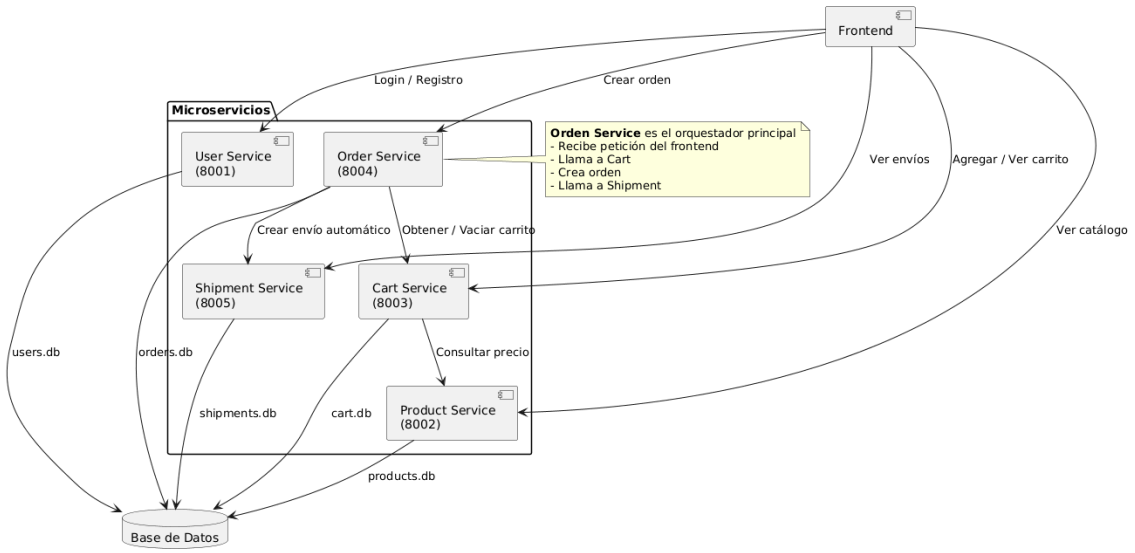

Figura 9: Arquitectura General del Sistema

El sistema utiliza Docker Compose para orquestar los cinco microservicios y sus respectivas bases de datos SQLite. La comunicación entre servicios se realiza mediante HTTP/REST y todos comparten una red interna llamada `ecommerce-net`.

11. Aprendizajes

Durante el desarrollo de este proyecto se adquirieron los siguientes conocimientos:

- Diseño e implementación de arquitecturas basadas en microservicios
- Comunicación entre servicios mediante APIs REST y manejo de tokens JWT
- Uso de Docker y Docker Compose para orquestación de contenedores
- Integración de FastAPI con SQLAlchemy y Pydantic
- Manejo de CORS y problemas comunes en aplicaciones distribuidas
- Buenas prácticas en el diseño de endpoints y validación de datos
- Depuración de errores en entornos multi-contenedor

12. Conclusiones

La implementación de este proyecto permitió comprender las ventajas y desafíos de trabajar con microservicios. Aunque aumenta la complejidad inicial en comparación con una aplicación monolítica, ofrece una mejor escalabilidad, mantenimiento y organización del código.

Se logró integrar exitosamente cinco servicios independientes con un frontend funcional, completando un flujo real de e-commerce. Este proyecto sirve como base sólida para futuras expansiones, como la incorporación de pagos, notificaciones y mayor robustez en la gestión de errores.

Aaron Rodrigo Ramos Reyes
Eduardo Hernández Soto
Diego Sebastian Reyes García
Cuajimalpa, Ciudad de México
30 de marzo de 2026